

南 阳 理 工 学 院

本科生毕业设计(论文)

计算机与软件学院

计算机科学与技术

XXX

XXX

完成日期 ____2026__05__月

南阳理工学院本科生毕业设计 (论文)

基于 Spark 的豆瓣影视数据分析系统的设计与实现

**Design and Implementation of a Douban Movie and TV Data
Analysis System Based on Spark**

总计：毕业设计(论文) 49 页

表格：9 个

插图：30 幅

南 阳 理 工 学 院 本 科 毕 业 设 计(论文)

基于 Spark 的豆瓣影视数据分析系统的设计与实现

Design and Implementation of a Douban Movie and TV Data Analysis System Based on Spark

学 院(系) : 计算机与软件学院

专 业 : 计算机科学与技术

学 生 姓 名 : XXX

学 号 : XXXXXXXXXXXX

指导教师(职称) : XXX (讲师)

评 阅 教 师 : XXX

完 成 日 期 : 2026 年 05 月

南阳理工学院

Nanyang Institute of Technology

基于 Spark 的豆瓣影视数据分析系统的设计与实现

计算机科学与技术专业 XXX

[摘 要]随着豆瓣影视数据规模的持续增长，用户对影视作品的评分、评论、题材分布以及导演与地区特征等信息的分析需求不断提升。为了实现对豆瓣影视数据的统一采集、清洗、统计分析、情感判断、类型预测与相似推荐，系统设计并实现了一套基于 Spark 的豆瓣影视数据分析系统。系统以 Spark 分布式计算框架为数据处理核心，结合 DataFrame 与 Spark SQL 完成批量清洗、统计汇总和特征构建，再通过 Python 数据分析技术、Flask Web 框架以及 ECharts 可视化组件构建从数据处理到结果展示的一体化流程。在功能实现上，系统提供了数据概览、影视浏览、评分分析、类型分析、导演分析、地区分析、年份趋势、情感分析、词云展示、类型预测和影视推荐等模块，能够较为完整地反映豆瓣影视数据的统计特征与内容特征。测试部分基于 4371 部影视数据完成了文本分类对比测试，结果表明在当前数据集上，支持向量机模型的综合性能较优；在改进测试中，结合 jieba 分词与结构化特征后，模型的 F1 值进一步提升，说明特征工程对影视类型识别具有明显作用。系统为影视数据的教学演示、推荐分析和可视化展示提供了一个可落地的参考系统，也为后续扩展到更大规模的 Spark 集群分析奠定了基础。

[关键词] Spark；豆瓣影视；数据分析；机器学习；可视化

Design and Implementation of a Douban Movie and TV Data Analysis System Based on Spark

Computer Science and Technology Major XXX

Abstract: With the continuous growth of Douban movie and TV data, the demand for unified analysis on ratings, reviews, genre distribution, director features and regional characteristics is increasing. This study presents a Douban movie and TV data analysis system based on Spark. The system uses Spark as the core data processing engine and combines DataFrame and Spark SQL for batch cleaning, aggregation and feature construction. It integrates data acquisition, cleaning, statistical analysis, sentiment analysis, genre prediction and similarity recommendation into a single workflow. The workflow further combines Python data analytics, the Flask web framework and ECharts for interactive visualization. The implemented system provides modules for data overview, movie browsing, rating analysis, genre analysis, director analysis, regional analysis, yearly trends, sentiment analysis, word clouds, genre prediction and movie recommendation. Tests on 4,371 movie records show that SVM achieves the best overall performance among the tested classifiers, and the improved feature set further increases the weighted F1 score, demonstrating the benefit of feature engineering for movie genre classification. The system offers a practical reference for teaching demonstration, recommendation analysis and visual analytics, and can be extended to larger Spark clusters in future work.

Keywords: Spark; Douban movies; data analysis; machine learning; visualization

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.2.1 影视数据分析与推荐研究	1
1.2.2 文本分类与情感分析研究	2
1.3 系统组织结构	2
2 系统关键技术	3
2.1 Spark 分布式计算框架	3
2.1.1 DataFrame 与 Spark SQL	3
2.1.2 Spark 批处理任务组织	3
2.2 Flask 与 Web 服务框架	3
2.3 数据处理与机器学习技术	4
2.3.1 Pandas 与 NumPy	4
2.3.2 scikit-learn 分类模型	4
2.4 中文分词与情感分析技术	4
2.4.1 jieba 中文分词	5
2.4.2 SnowNLP 情感分析	5
2.5 可视化与存储技术	5
2.5.1 ECharts 交互可视化	5
2.5.2 SQLite 与 SQLAlchemy	6
3 系统分析	7
3.1 可行性分析	7
3.1.1 技术可行性	7
3.1.2 经济可行性	7
3.1.3 操作可行性	7
3.2 用户需求分析	7
3.2.1 基础数据需求	8
3.2.2 专题分析需求	8
3.2.3 智能分析需求	8
3.3 系统用例分析	8
3.4 非功能需求分析	10
4 系统设计	11
4.1 系统总体架构设计	11
4.1.1 数据源层与 Spark 处理层	11
4.1.2 服务应用层与前端展示层	12
4.2 功能模块设计	12

4.2.1 基础数据模块设计.....	12
4.2.2 专题分析模块设计.....	12
4.2.3 智能分析模块设计.....	12
4.3 系统流程设计.....	13
4.3.1 整体业务流程设计.....	13
4.3.2 数据采集与预处理流程设计.....	13
4.3.3 智能分析流程设计.....	14
4.4 数据库与文件存储设计.....	15
4.4.1 数据库概念结构设计.....	15
4.4.2 数据库物理结构设计.....	17
5 系统实现.....	20
5.1 数据采集与预处理的实现.....	20
5.1.1 虚拟机网络配置实现.....	20
5.1.2 网络连通性测试实现.....	21
5.1.3 Spark 环境启动实现.....	21
5.1.4 数据读取与清洗实现.....	22
5.1.5 特征工程实现.....	23
5.2 基础数据管理功能的实现.....	24
5.2.1 Flask 服务运行实现.....	24
5.2.2 数据概览实现.....	25
5.2.3 影视浏览实现.....	26
5.3 专题分析功能的实现.....	27
5.3.1 评分分析实现.....	27
5.3.2 类型分析实现.....	28
5.3.3 导演分析实现.....	28
5.3.4 地区分析实现.....	29
5.3.5 年份趋势实现.....	30
5.4 智能分析功能的实现.....	31
5.4.1 情感分析实现.....	31
5.4.2 词云分析实现.....	32
5.4.3 模型对比实现.....	33
5.4.4 类型预测实现.....	35
5.4.5 影视推荐实现.....	35
6 系统测试.....	37
6.1 测试环境与测试方法.....	37
6.2 系统功能用例测试.....	37
6.3 模型效果测试.....	38
6.3.1 分类模型对比.....	38
6.3.2 改进测试分析.....	38

6.4 本章小结.....	38
7 总结和展望.....	39
7.1 总结.....	39
7.2 展望.....	39
参考文献.....	40
致谢.....	41

1 绪论

1.1 研究背景与意义

随着互联网视频平台和影视评分社区的不断发展，海量影视作品、用户评分和评论内容被持续沉淀下来，形成了具有较高分析价值的数据资源。豆瓣作为国内较有代表性的影视评分与评论平台，聚合了大量关于影片类型、导演、主演、上映年份、地区、评分人数以及用户评论的信息，既能反映作品本身的内容特征，也能反映观众的主观偏好和审美趋势。

对豆瓣影视数据进行系统化分析，可以从评分分布、题材热度、导演特征、地区差异和评论情感等维度揭示影视作品的内容特征与用户偏好，并为影片推荐、内容运营和数据分析教学提供支撑。将数据采集、清洗、建模、可视化和 Web 展示整合为完整信息系统，也能够体现数据分析方法在工程实践中的应用价值。

1.2 国内外研究现状

围绕影视数据分析、文本分类和推荐系统的研究已经形成了较为成熟的技术路线。国内关于 Spark 电影推荐系统的研究表明，分布式计算框架能够支撑影视数据的批量处理与推荐计算^[1]。相关系统实现工作进一步说明，Spark 与 Web 展示平台结合后，可以同时承担数据处理、推荐结果生成和页面展示任务^[2]。国外关于机器学习电影推荐的研究也说明，特征构建和算法选择会影响推荐质量^[3]。在模型方法方面，深度学习与 Spark 结合能够增强影视推荐系统的特征表达能力^[4]。基于 Spark 大数据处理的电影推荐系统研究则说明，影视数据分析系统需要同时关注数据清洗、模型训练和结果展示等环节^[5]。

从具体方向看，影视题材识别、评论情感分析和用户偏好预测是研究热点。无监督机器学习电影推荐系统研究表明，影片特征组织方式会直接影响推荐效果^[6]。现有工作中，传统机器学习方法如朴素贝叶斯、决策树和支持向量机仍然具有较好的基线价值，而在特征工程、类别平衡和模型调参之后，文本分类性能通常能够获得明显提升。与此同时，基于 Web 的分析系统越来越重视交互体验，希望把数据清洗、模型训练和结果可视化统一到一个平台中，便于用户直接浏览和验证分析结果。

1.2.1 影视数据分析与推荐研究

影视推荐类系统通常围绕作品元数据、用户评分和历史行为记录展开，通过统计分析和机器学习模型提取影片特征，再结合相似度计算或预测模型为用户提供候选影片。Hadoop 和 Spark 在 Web 系统推荐功能中的应用研究说明，推荐模块可以与 Web 服务形成稳定的数据调用关系^[7]。聚类挖掘智能推荐系统研究表明，Spark 平台适合承载批量聚合和用户偏好计算任务^[8]。其他大数据分析算法研究也说明，面向批量数据的算

法流程需要把数据预处理、特征计算和结果输出拆分为可复用环节^[9]。基于机器学习的电影推荐系统研究进一步验证了内容特征、协同过滤和相似度计算在推荐任务中的作用^[10]。

豆瓣数据相关研究为系统图表组织方式提供了直接参考。基于 Python 的豆瓣数据分析工作说明，公开平台数据适合通过采集、清洗和可视化形成分析结果^[11]。豆瓣影评情感分析研究表明，评论内容能够用于补充作品口碑判断^[12]。面向豆瓣电影的数据采集与分析可视化研究进一步说明，评分、类型、导演和地区等字段具有较高的展示价值^[13]。基于 Python 爬虫的豆瓣电影影评可视化分析也为评论数据处理和图表展示提供了参考^[14]。

1.2.2 文本分类与情感分析研究

评论内容的情感分析和题材分类是影视数据处理中最典型的自然语言处理任务之一。国外电影评论分类研究表明，机器学习方法可以用于影评语义分类并比较不同算法效果^[15]。基于文本挖掘的电影评论情感分析研究说明，影评内容经过分词和特征提取后能够用于情绪倾向判断^[16]。基于词典和弱标注信息的电影评论情感分析研究说明，轻量化文本方法在电影评论场景中具有较好的应用价值^[17]。中文评论情感分析方法研究则为情感得分、情感极性划分和文本特征处理提供了方法参考^[18]。

情感分析通常用于衡量评论内容中正向、负向和中性倾向，而文本分类更关注剧情简介或评论内容所对应的主题类别。系统在设计中同时保留了这两条分析路径：一方面通过 SnowNLP 对评论情绪进行快速判断，另一方面基于 TF-IDF、jieba 分词和结构化特征完成影视类型预测，从而让系统既能分析评论倾向，也能分析影片题材。

1.3 系统组织结构

系统围绕基于 Spark 的豆瓣影视数据分析系统展开，整体共分为七章。第一章介绍课题背景、研究意义以及国内外研究现状；第二章给出系统实现过程中实际使用的关键技术，包括 Spark 批处理、Flask 接口服务、Python 数据处理、中文分词、情感分析机器学习建模、可视化和轻量存储技术；第三章从可行性、用户需求、用例关系和非功能需求四个方面对系统进行分析；第四章详细给出系统总体架构、功能模块、流程和数据库与文件存储设计；第五章重点介绍系统主要功能模块的实现方法；第六章对系统功能和模型效果进行测试与评估；第七章总结全文并展望后续工作。

2 系统关键技术

2.1 Spark 分布式计算框架

Spark 是一种面向大规模数据处理的通用计算框架，具有内存计算、惰性执行和较强的容错能力。相比传统磁盘式批处理模式，Spark 能够在迭代计算和交互式分析场景下获得更好的性能，因此适合用于影视数据的批量读取、字段清洗、统计汇总和特征准备。系统使用 Spark 组织批处理链路，将原始影视数据转换为后续 Web 展示和模型分析可以复用的结构化结果。

本系统实际使用 Spark 读取影视数据文件，并对评分、年份、类型、地区和文本字段进行清洗转换。清洗后的结果再进入专题统计和智能分析流程。第二章仅保留与系统后续设计和实现直接对应的技术内容，避免把未在系统中体现的框架或模型写成泛泛的技术综述。

2.1.1 DataFrame 与 Spark SQL

DataFrame 为结构化数据分析提供了表格化接口，既保留字段关系的可读性，又能通过 Spark 优化器提升执行效率。在影视分析场景中，影片名称、评分、评分人数、类型、导演、国家、上映年份和简介等字段都具有明确含义，因此适合用 DataFrame 完成字段转换、过滤和聚合。

Spark SQL 进一步把结构化查询能力引入数据处理过程，使统计汇总、分组查询和条件筛选更加简洁。系统在后续实现中围绕“读取数据、清洗字段、注册临时视图、输出统计结果”的流程组织 Spark 任务，使数据处理逻辑与 Web 展示逻辑保持分离。

2.1.2 Spark 批处理任务组织

系统使用 Spark 批处理任务完成影视数据清洗和中间结果生成。任务启动后先读取数据源，再对重复记录、缺失字段和异常评分进行处理，随后完成类型拆分、年份提取和文本字段标准化。该流程能够为首页统计、专题分析、类型预测和推荐计算提供一致的数据基础。

2.2 Flask 与 Web 服务框架

Flask 是一个轻量级 Python Web 框架，适合构建结构清晰、模块边界明确的中小型系统。本系统中的可视化平台采用 Flask 组织路由、登录认证和接口服务，将数据概览、影视浏览、评分分析、类型分析、导演分析、地区分析、年份趋势、情感分析、词云分析、模型对比、类型预测和影视推荐等功能拆分为独立页面和 API 接口，从而提升系统维护性。

在前端呈现方面，系统结合 Bootstrap 和 ECharts 完成页面布局与图表渲染。Bootstrap 负责基础栅格、按钮、表单和响应式布局，ECharts 则用于柱状图、折线图、热力图、散点图和词云图等交互式图表展示，能够让用户更直观地观察影视数据中的规律。

2.3 数据处理与机器学习技术

数据处理阶段主要依赖 Pandas 和 NumPy 完成清洗、转换与统计，机器学习阶段则借助 scikit-learn 完成特征向量化、模型训练、交叉验证和结果评估。Pandas 适合处理表格型影视元数据，可以快速完成空值过滤、去重、字段拆分和统计汇总；NumPy 则在数值计算、向量运算和指标汇总中提供基础支撑。机器学习算法在电影推荐系统中的应用研究表明，多模型对比有助于判断算法适配性^[19]。

scikit-learn 是系统模型测试的核心工具。针对影视类型预测任务，系统比较了朴素贝叶斯、决策树和线性支持向量机等模型，并通过 TF-IDF 向量化、中文分词、类别重构、结构化特征融合和参数调节等方式改进测试效果。Spark 与电影推荐工程应用研究也说明，数据处理框架与模型分析流程需要在系统设计中形成清晰衔接^[20]。

2.3.1 Pandas 与 NumPy

Pandas 擅长对影视元数据进行表格化处理，例如将评分、评分人数和上映年份转换为数值类型，将类型字段拆分为首要类型和全部类型，将国家与地区字段拆分为多个维度。NumPy 则在统计分布、均值、中位数和概率计算中发挥作用，使系统能够快速完成评分区间统计和模型结果汇总。

在本系统中，数据清洗后的字段会被进一步送入统计分析模块和机器学习模块。这样的处理方式既减少了重复计算，也便于在 Web 层直接复用分析结果。

2.3.2 scikit-learn 分类模型

scikit-learn 提供了从特征工程到模型评估的完整工具链，能够很好地支持文本分类任务。系统在类型预测测试中采用 TF-IDF 表示影视剧情简介，并使用朴素贝叶斯、决策树和线性支持向量机进行分类，对比不同模型在准确率、精确率、召回率和加权 F1 值上的表现。

从测试结果看，线性支持向量机在影视类型识别任务中具有较好的综合性能，而经过特征增强和类别重构之后，模型表现仍能继续改善，这说明传统机器学习在中等规模文本分类任务中仍然具有稳定的应用价值。

2.4 中文分词与情感分析技术

中文文本分析面临的首要问题是缺少天然分隔符，因此需要先进行分词，再进行特征提取和分类。jieba 是中文分词领域使用较广的开源工具，支持精确模式、全模式

和搜索引擎模式，适合对影视简介和评论内容进行快速处理。系统在分词过程中结合停用词表，对常见虚词和无意义词进行过滤，以提高关键词提取和分类建模的有效性。

SnowNLP 则适合对中文短文本进行情感倾向判断，能够给出 0 到 1 之间的情感得分。系统使用 SnowNLP 对豆瓣评论进行快速情感分析，并按照阈值划分为正面、中性和负面三类，使系统能够从主观情绪角度补充对影视作品的评价视图。

2.4.1 jieba 中文分词

在影视剧情简介处理中，分词质量直接影响后续的 TF-IDF 特征表达效果。jieba 能够较稳定地识别中文词语边界，尤其适合“剧情简介较长、名词短语较多”的文本场景。结合自定义停用词表之后，可以进一步剔除“电影”“故事”“影片”等低区分度词语，突出真正能反映题材特征的关键词。

系统在改进测试中引入 jieba 分词后，模型对中文语义边界的捕获能力更强，说明面向中文影视文本的分类任务，分词策略是提升效果的关键步骤之一。

2.4.2 SnowNLP 情感分析

SnowNLP 提供了较轻量的中文情感分析能力，适合在网页系统中做即时情绪判断。用户提交评论内容后，系统可以快速返回情感得分和情感标签，并结合评论样本的整体分布展示影片口碑特征。

SnowNLP 属于轻量级中文文本情感分析工具，部署和调用成本较低，能够满足原型系统中评论情感快速判定的需求。系统将其作为情感分析模块的基础方案，并通过阈值规则把情感得分转换为页面可展示的情感标签。

2.5 可视化与存储技术

ECharts 是系统 Web 前端的主要可视化工具，能够通过折线图、柱状图、热力图、散点图和词云图等组件展示分析结果。对于影视数据这类具有明显多维结构的数据集，可视化能够把抽象的统计结果转化为直观图形，使用户更容易观察评分分布、类型热度、导演影响力和年份变化趋势。

在存储方面，系统采用 SQLite 保存用户登录和认证信息，使用 SQLAlchemy 统一处理数据库对象关系；影视元数据则保留在 Excel 数据文件中，测试结果以 JSON 形式保存，便于后续复现和对比。这样的设计兼顾了原型系统的轻量性与数据分析过程的可追溯性。

2.5.1 ECharts 交互可视化

ECharts 的优势在于图表类型丰富、交互能力强、配置灵活，能够满足系统中多种图表展示需求。系统在数据概览、评分分析、类型分析、导演分析、地区分析和年份趋势等页面中均使用 ECharts 渲染图表，并结合颜色渐变和交互提示增强阅读体验。

在系统文档编写中，可视化结果除了用于页面展示，也可以直接作为测试依据嵌入正文，从而让系统效果具有更强的说服力。

2.5.2 SQLite 与 SQLAlchemy

SQLite 是一种轻量级嵌入式数据库，适合单机部署和教学演示场景。系统仅将用户账号、密码哈希、邮箱和登录时间等少量信息存入数据库，而将更大体量的影视数据保留在文件系统或分析结果文件中，以降低系统部署复杂度。

SQLAlchemy 则帮助系统以面向对象的方式管理数据表结构和数据库访问逻辑，使登录注册、密码校验和账号持久化的实现更加清晰。

3 系统分析

3.1 可行性分析

可行性分析从技术条件、经济投入和用户操作三个角度展开，重点判断系统在当前开发环境下完成数据处理、Web 展示和智能分析功能的可实施性。

3.1.1 技术可行性

从技术角度看，系统涉及的数据采集、清洗、分析、建模和可视化均可以使用成熟的开源工具完成，所需技术栈与当前主流 Python 数据分析生态高度匹配，开发风险较低。Spark 负责批量数据处理与统计汇总，Flask 负责页面路由和接口服务，ECharts 负责图表展示，scikit-learn、jieba 与 SnowNLP 负责智能分析相关处理。上述工具均具备较成熟的使用案例和资料基础，能够支撑系统按计划完成。系统技术实现由 Spark 数据处理、Web 服务、机器学习和可视化展示四部分组成，各部分之间通过 CSV、Excel、JSON 和 HTTP 接口衔接，技术路线清晰。

3.1.2 经济可行性

从经济角度看，系统主要依赖开源框架、本地数据文件和普通开发环境，不需要购买商业软件或额外硬件。若后续数据规模继续扩大，可以逐步迁移到多节点 Spark 集群，而当前阶段采用单机 Spark 与本地文件存储即可满足开发、调试和展示要求。影视数据、模型测试结果和页面截图均保存在项目目录下，便于重复构建和后续维护，能够在较低成本下完成从数据处理到系统展示的完整闭环。

3.1.3 操作可行性

从操作角度看，系统采用浏览器访问方式，用户登录后即可通过侧边栏进入基础数据、专题分析和智能分析页面。系统主要交互为查询、筛选、查看图表、输入文本和获取推荐结果，操作流程短、学习成本低，适合面向影视数据分析场景进行展示和验证。页面操作以点击、输入和筛选为主，不要求用户掌握 Spark 或 Python 命令，因此具备较好的可操作性。

3.2 用户需求分析

系统用户需求围绕“基础数据、专题分析、智能分析”三条主线展开。基础数据侧重数据概览、影视浏览、用户登录和数据接口；专题分析侧重评分、类型、导演、地区和年份等统计维度；智能分析侧重情感分析、词云分析、模型对比、类型预测和影视推荐。三类功能共同支撑豆瓣影视数据从原始数据到分析结论的转化过程。

从用户角度看，系统需要降低影视数据分析的操作门槛。用户不需要直接执行 Spark 命令或模型训练脚本，而是通过浏览器页面完成登录、查询、筛选、图表查看和

智能分析操作。系统后台负责把清洗结果、模型指标和推荐结果封装为接口数据，保证用户在页面中即可获得分析结果。

3.2.1 基础数据需求

基础数据需求包括用户登录、数据概览、影视浏览和接口调用。用户需要通过账号进入系统，查看影视总量、平均评分、评分人数、类型数量等总体指标，并能够按照片名、导演、类型或年份快速定位影视记录。前端页面需要通过接口获取分页列表和统计数据，保证页面展示与后端分析结果保持一致。

3.2.2 专题分析需求

专题分析需求包括评分分析、类型分析、导演分析、地区分析和年份趋势分析。用户需要从评分分布、类型热度、导演作品数量、地区分布和年度变化等角度理解影视数据特征。各专题页面应提供图表展示和必要文字说明，使用户能够直接观察数据规律。

3.2.3 智能分析需求

智能分析需求包括评论情感分析、词云分析、模型对比、影视类型预测和相似影片推荐。系统需要将评论内容、剧情简介和结构化字段转化为模型可处理的特征，并输出情感标签、关键词、模型指标、预测类别和推荐列表，从而体现系统的数据挖掘能力。

3.3 系统用例分析

系统用例分析用于进一步说明用户与各功能之间的关系。根据系统功能边界，系统参与者包括普通用户和后台数据处理流程。普通用户负责触发页面操作，后台流程负责提供经过清洗与训练后的结果数据。主要用例包括登录系统、浏览影视数据、查看专题分析、提交评论分析、执行类型预测和获取相似推荐。

系统用户用例图如图 3-1 所示。图中以普通用户为主要参与者，将数据查看、专题统计和智能分析功能纳入统一系统边界，同时把文本预处理、特征向量化和相似度计算作为智能分析用例的包含环节。

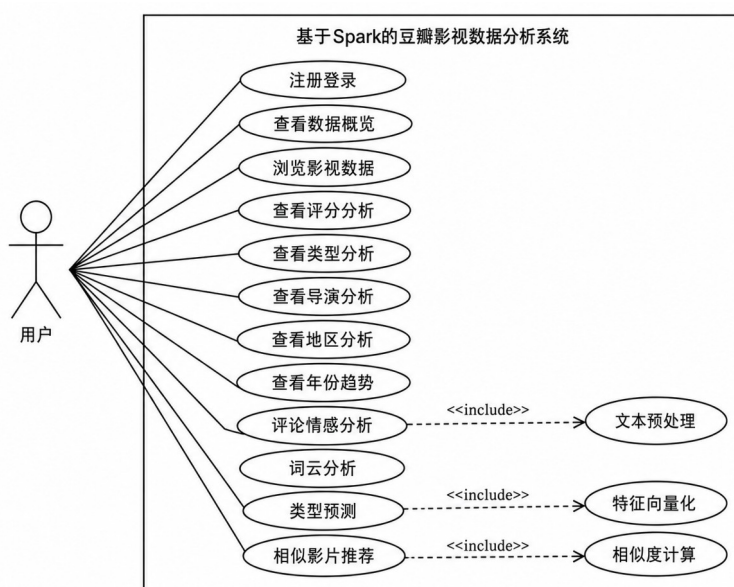


图 3-1 系统用户用例图

为便于进一步说明核心用例的执行条件和返回结果，表 3-1 从参与者、前置条件、基本流程和预期结果等方面对主要用例进行整理。基础数据模块提供系统入口，专题分析模块提供统计结论，智能分析模块提供模型化判断；三类用例之间存在数据依赖关系，影视浏览结果为分析页面提供对象基础，数据清洗和特征工程为类型预测与推荐提供输入，测试结果文件为模型对比页面提供展示依据。

表 3-1 系统主要用例说明表

用例名称	参与者	前置条件	基本流程	预期结果
登录系统	普通用户	用户已注册账号	输入用户名和密码，提交登录请求	进入系统首页并保持会话状态
浏览影视数据	普通用户	用户已登录	输入关键词或选择类型条件，查看分页结果	返回符合条件的影视列表
查看专题分析	普通用户	分析结果已生成	进入评分、类型、导演、地区或年份页面	图表正常渲染并显示统计结果
执行情感分析	普通用户	用户已登录	输入评论内容并提交分析	返回情感得分和情感标签
进行类型预测	普通用户	模型文件已加载	输入剧情简介并提交预测	返回预测类型和置信度信息
获取影视推荐	普通用户	影视特征向量已生成	选择或输入目标影视作品	返回相似影片推荐列表

3.4 非功能需求分析

系统非功能需求主要包括性能、稳定性、安全性和可维护性。性能方面，系统需要在本地环境下快速响应数据筛选和图表渲染请求；稳定性方面，系统需要在数据缺失、空值异常或文本字段不规范时保持可运行状态；安全性方面，用户密码应采用哈希方式保存，登录接口应限制未授权访问；可维护性方面，代码应拆分为数据服务、视图层、接口层和模型服务层，便于后续扩展。

考虑到系统以影视数据分析和功能展示为主要目标，非功能需求不追求高并发，而是强调结构清晰、结果可靠、页面稳定和后续可扩展。该定位与当前系统规模相匹配。

4 系统设计

4.1 系统总体架构设计

系统采用“数据源层、Spark 处理层、服务应用层、前端展示层”的分层架构。数据源层保存豆瓣影视数据、评论样本、模型测试数据和用户认证数据；Spark 处理层负责数据读取、清洗、统计汇总、特征构建和结果生成；服务应用层由 Flask 提供路由、登录认证和 API 接口；前端展示层使用 Bootstrap 和 ECharts 组织页面与图表。

系统总体架构如图 4-1 所示。该架构将数据处理任务与 Web 展示任务分离，既便于说明 Spark 在系统中的核心作用，也便于后续将本地批处理流程迁移到更大规模的数据处理环境。

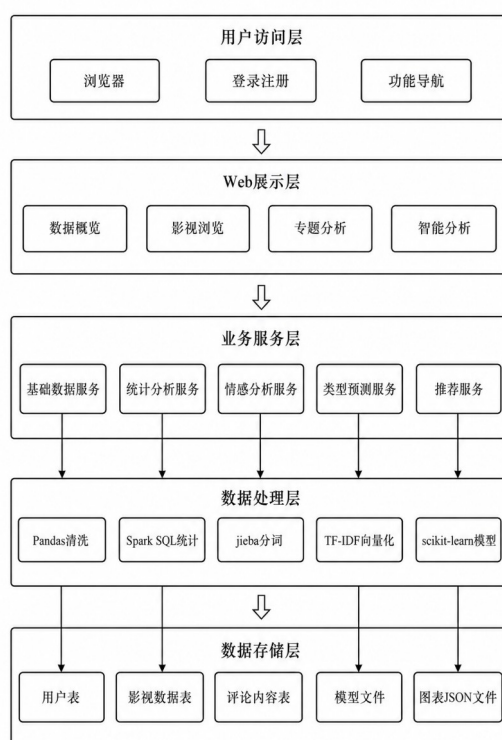


图 4-1 基于 Spark 的系统总体架构图

4.1.1 数据源层与 Spark 处理层

数据源层主要包括 douban_movies.xlsx、评论样本数据、模型测试结果 JSON 和 SQLite 用户数据库。Spark 处理层先读取影视数据，再完成字段标准化、空值过滤、类型拆分、统计汇总和特征提取。处理结果以 JSON 或内存对象形式传递给 Web 服务层，供页面图表和模型功能调用。

4.1.2 服务应用层与前端展示层

服务应用层由 Flask 组织用户认证、页面路由和接口服务，前端展示层通过模板页面、Bootstrap 样式和 ECharts 图表完成交互展示。用户在浏览器中完成查询、筛选和智能分析操作，系统通过接口返回结构化结果，形成前后端协同流程。

4.2 功能模块设计

系统功能模块图如图 4-2 所示。图中仅保留三大模块及其子功能，分别对应基础数据、专题分析和智能分析，使三级标题与模块划分保持一致。

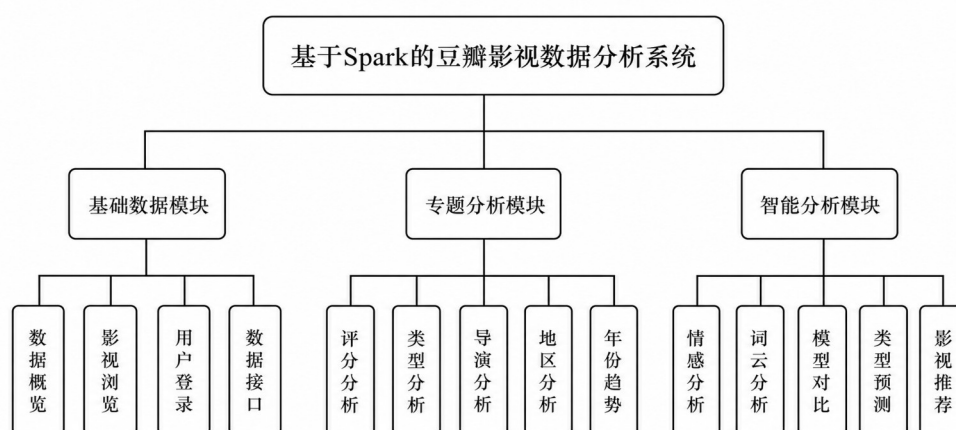


图 4-2 系统功能模块图

4.2.1 基础数据模块设计

基础数据模块负责支撑系统最基本的数据访问能力。用户登录后进入首页，系统展示影视总量、平均评分、评分人数、类型数量等关键指标；影视浏览页面支持关键词搜索、类型筛选、排序和分页；数据接口负责向前端返回 JSON 格式结果。该模块是后续专题分析与智能分析的入口。

4.2.2 专题分析模块设计

专题分析模块围绕影视元数据展开统计。评分分析用于观察评分分布和评分人数关系，类型分析用于展示题材热度和平均评分，导演分析用于统计创作者作品数量和评分水平，地区分析用于呈现不同地区影片占比，年份趋势用于展示影视作品数量和评分随时间变化的情况。

4.2.3 智能分析模块设计

智能分析模块围绕文本和模型输出展开。情感分析根据评论内容判断情绪倾向，词云分析展示高频关键词，模型对比展示不同分类算法效果，类型预测根据剧情简介输出题材类别，影视推荐根据文本向量相似度返回候选影片。该模块体现系统在数据挖掘和辅助决策方面的工作量。

4.3 系统流程设计

系统流程设计包括整体业务流程、数据采集与预处理流程、智能分析核心流程三个层面。整体业务流程说明用户进入系统后的功能选择和接口调用关系，数据采集与预处理流程说明原始影视数据如何转换为可分析数据，智能分析流程说明评论情感、类型预测和相似推荐的处理步骤。

4.3.1 整体业务流程设计

整体业务流程从用户访问系统开始。用户完成登录校验后进入数据概览页面，并根据需要选择基础数据、专题分析或智能分析功能。对于普通统计类页面，系统调用数据服务返回图表数据；对于情感分析、类型预测和推荐页面，系统调用模型服务返回分析结果。若用户输入为空或参数不合法，系统返回提示信息并保持页面可用。

系统整体流程如图 4-3 所示。该流程体现了用户操作、接口调用、结果读取和页面展示之间的基本顺序。

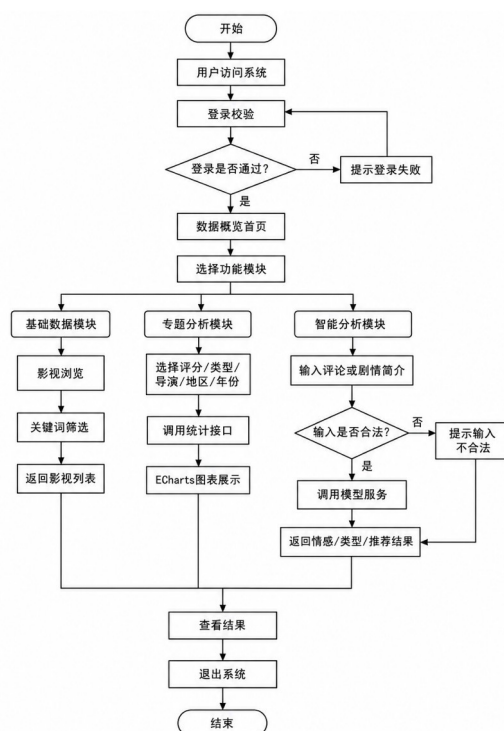


图 4-3 系统整体流程图

4.3.2 数据采集与预处理流程设计

数据采集与预处理流程以 `douban_movies.xlsx` 为输入，先读取影视数据集并完成字段完整性检查，再对缺失值、重复记录和异常评分进行过滤。清洗后的数据按照评分、

年份、类型和地区等字段进行标准化处理，同时对评论内容和剧情简介进行分词、停用词过滤和 TF-IDF 特征构建，最终生成统计分析和智能分析可复用的数据文件。

数据采集与预处理流程如图 4-4 所示。该流程把原始数据整理为后续页面展示和模型分析可复用的中间结果。

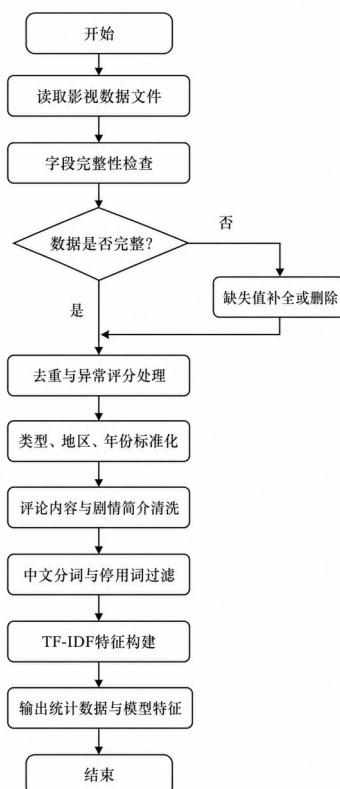


图 4-4 数据采集与预处理流程图

4.3.3 智能分析流程设计

智能分析流程从用户输入评论内容或剧情简介开始，系统进行输入合法性校验后，完成文本清洗、jieba 分词和 TF-IDF 向量化。根据用户选择的任务，系统分别调用情感分析、类型预测或相似推荐服务，并将情感标签、预测类别或推荐列表返回前端。该流程能够把文本处理、模型调用和结果展示连接成闭环。

智能分析核心流程如图 4-5 所示。该流程覆盖情感分析、类型预测和相似推荐三类智能分析任务。

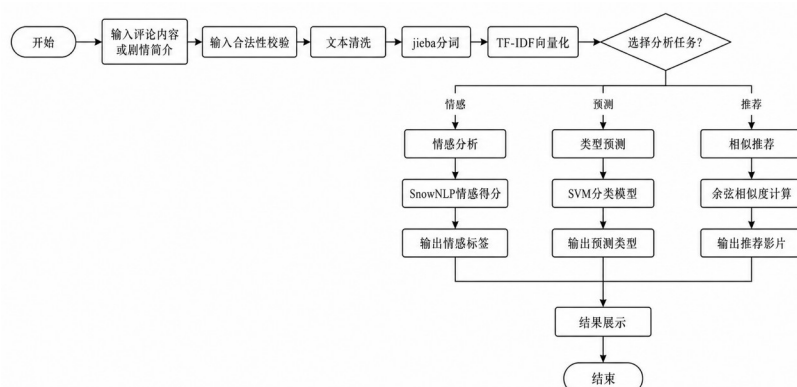


图 4-5 智能分析核心流程图

4.4 数据库与文件存储设计

系统采用“轻量数据库+文件化分析结果”的混合存储方式。用户登录信息保存到 SQLite 数据库，影视元数据保存在 Excel 文件中，评论样本与模型结果保存在 CSV 或 JSON 文件中。该方案能够兼顾部署简单、结果可复现和数据结构清晰等要求，也符合系统当前以数据分析和展示为核心的定位。

4.4.1 数据库概念结构设计

数据库概念结构设计关注系统中主要数据对象及其联系。根据系统功能，主要实体包括用户、影视数据、评论数据、分析结果和推荐记录五类。用户实体负责身份认证，影视数据实体负责保存影片基本属性，评论数据实体负责保存文本评价内容，分析结果实体负责承载统计图表和模型指标，推荐记录实体负责保存相似推荐关系。

实体属性如图 4-6 至图 4-10 所示。由于影视元数据来自文件而非完整关系型数据库，系统将影片编号、片名、类型、导演、地区、评分、年份和简介作为影视数据实体的核心属性；分析结果和推荐记录虽然以文件结果为主，也需要在概念结构中体现其与影视数据的关系。

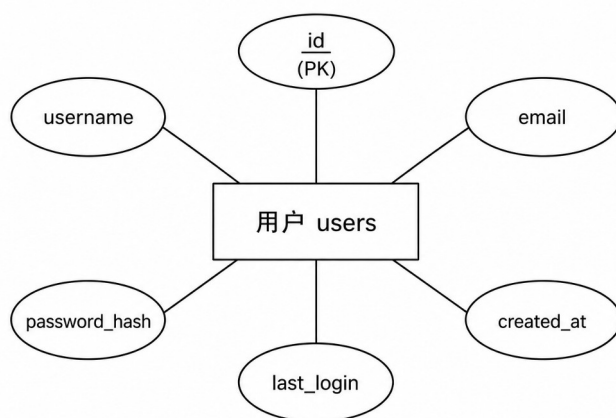


图 4-6 用户实体属性图

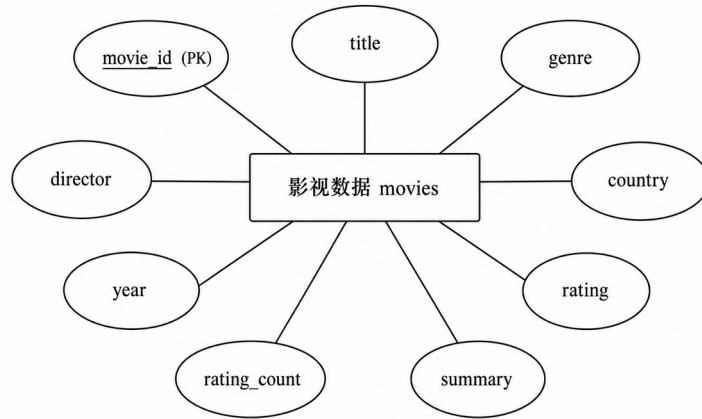


图 4-7 影视数据实体属性图

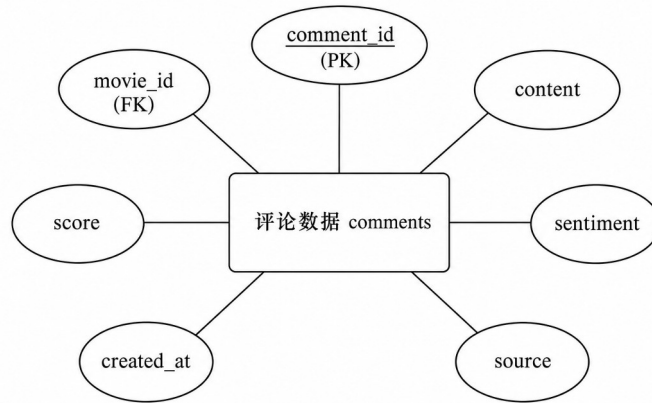


图 4-8 评论数据实体属性图

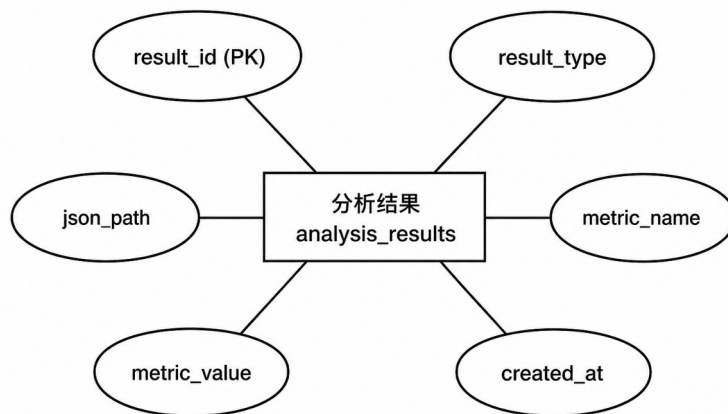


图 4-9 分析结果实体属性图

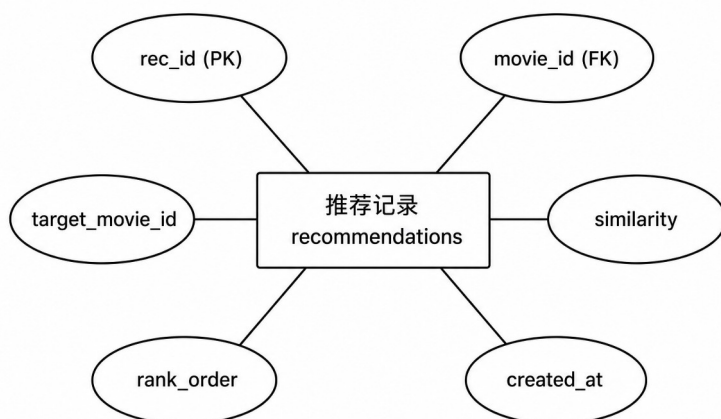


图 4-10 推荐记录实体属性图

系统总体 E-R 图如图 4-11 所示。图中用户与影视数据之间存在浏览和查询关系，影视数据与评论数据、分析结果和推荐记录之间形成一对多或关联关系。

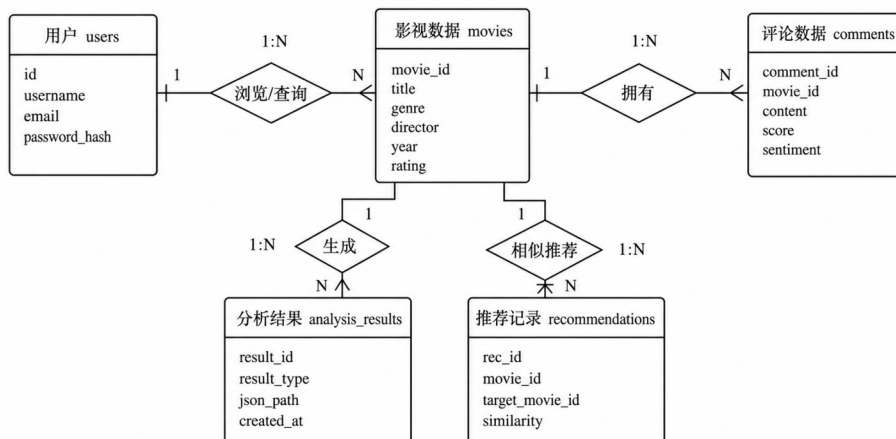


图 4-11 系统总体 E-R 图

4.4.2 数据库物理结构设计

数据库物理结构设计用于说明主要实体在系统中的字段组织方式。用户表采用 SQLite 持久化，影视数据、评论数据、分析结果和推荐记录以文件化方式保存，但仍按照数据表字段进行描述，便于说明字段来源、类型和用途。

表 4-1 用户表(users)

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY	用户编号，自增主键
username	VARCHAR(64)	UNIQUE, NOT NULL	登录用户名
email	VARCHAR(128)	UNIQUE	用户邮箱
password_hash	VARCHAR(255)	NOT NULL	密码哈希值
created_at	DATETIME	DEFAULT	账号创建时间

		CURRENT_TIMESTAMP	
		MP	
last_login	DATETIME	NULL	最后登录时间

表 4-2 影视数据表 (movies)

字段名	类型	约束	说明
movie_id	INTEGER	PRIMARY KEY	影视编号或文件行号
title	VARCHAR(255)	NOT NULL	影视名称
genre	VARCHAR(128)	NOT NULL	影视类型，支持多类型拆分
director	VARCHAR(255)	NULL	导演或主创信息
country	VARCHAR(128)	NULL	制片国家或地区
year	INTEGER	NULL	上映年份
rating	DOUBLE	0-10	豆瓣评分
rating_count	INTEGER	NULL	评分人数
summary	TEXT	NULL	剧情简介或文本特征来源

表 4-3 评论数据表 (comments)

字段名	类型	约束	说明
comment_id	INTEGER	PRIMARY KEY	评论编号
movie_id	INTEGER	FOREIGN KEY	关联影视编号
content	TEXT	NOT NULL	评论文本内容
score	DOUBLE	NULL	评论或情感得分
sentiment	VARCHAR(32)	NULL	情感标签
source	VARCHAR(64)	NULL	评论来源
created_at	DATETIME	NULL	评论采集或写入时间

表 4-4 分析结果表 (analysis_results)

字段名	类型	约束	说明
result_id	INTEGER	PRIMARY KEY	分析结果编号
result_type	VARCHAR(64)	NOT NULL	评分、类型、导演、地区、年份或模型指标
json_path	VARCHAR(255)	NOT NULL	结果文件路径
metric_name	VARCHAR(64)	NULL	指标名称
metric_value	DOUBLE	NULL	指标数值
created_at	DATETIME	NULL	结果生成时间

表 4-5 推荐记录表 (recommendations)

字段名	类型	约束	说明
rec_id	INTEGER	PRIMARY KEY	推荐记录编号
movie_id	INTEGER	FOREIGN KEY	当前影视编号

target_movie_id	INTEGER	FOREIGN KEY	被推荐影视编号
similarity	DOUBLE	NOT NULL	余弦相似度
rank_order	INTEGER	NOT NULL	推荐排序
created_at	DATETIME	NULL	推荐结果生成时间

表 4-6 基础文件结构与存储方式

文件或目录	数据内容	存储方式	用途
data/ douban_movies.xlsx	影视名称、评分、类型、导演、地区、简介等字段	Excel 文件	作为基础影视数据源
data/ experiment_results.json	模型准确率、召回率、F1 值和混淆矩阵	JSON 文件	支撑模型对比页面
images/ system_screenshots/	系统运行界面截图	PNG 图片	支撑系统实现章节展示
images/final_figures/	运行结果和整理后的图像	PNG 图片	支撑系统运行说明

通过上述设计，系统既保留了登录认证所需的关系型数据表，也说明了影视数据、评论内容、模型结果和推荐结果在文件系统中的结构化组织方式，避免把全部分析数据强行写入数据库。

5 系统实现

5.1 数据采集与预处理的实现

数据采集与预处理是系统实现的起点。系统将豆瓣影视元数据保存为 `douban_movies.xlsx`，并通过 Spark 或 Pandas 读取为结构化数据对象。读取后，系统对评分、评分人数、上映年份、片长、类型和地区等字段进行清洗与转换，同时对剧情简介和评论内容进行分词与向量化处理。该过程保证后续统计分析、模型预测和相似推荐能够使用统一的数据格式。

5.1.1 虚拟机网络配置实现

为保证 Spark 运行环境能够被本机访问，系统先在虚拟机中配置静态 IP、主机名和基础网络参数。虚拟机静态 IP 与主机名配置如图 5-1 所示。

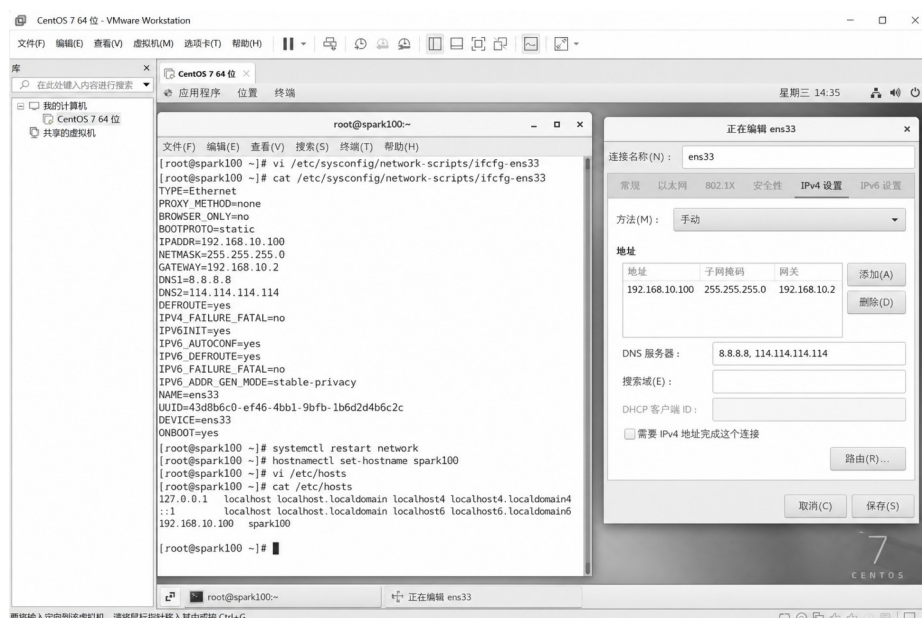


图 5-1 虚拟机静态 IP 与主机名配置

虚拟机网络配置功能实现的核心代码如下：

```
# /etc/sysconfig/network-scripts/ifcfg-ens33
TYPE=Ethernet
BOOTPROTO=static
NAME=ens33
DEVICE=ens33
ONBOOT=yes
IPADDR=192.168.10.100
NETMASK=255.255.255.0
GATEWAY=192.168.10.2
DNS1=8.8.8.8

# 修改主机名并重启网络服务
hostnamectl set-hostname spark100
systemctl restart network
```

5.1.2 网络连通性测试实现

网络配置完成后，系统通过主机与虚拟机之间的连通性测试确认开发环境可用。虚拟机网络连通性测试如图 5-2 所示。

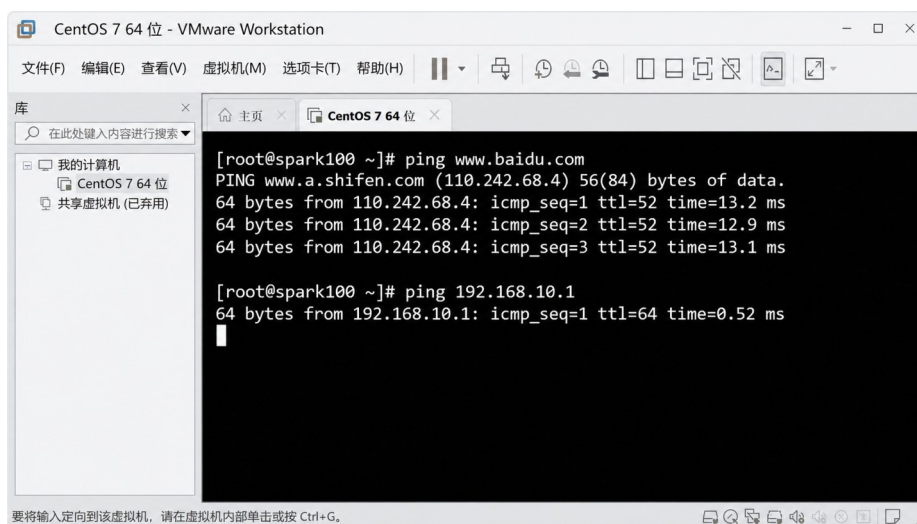


图 5-2 虚拟机网络连通性测试

网络连通性测试功能实现的核心代码如下：

```
# 测试虚拟机到外网的连通性
ping -c 4 www.baidu.com

# 测试虚拟机与宿主机网关的连通性
ping -c 4 192.168.10.1

# 查看当前网卡地址，确认静态 IP 生效
ip addr show ens33 | grep inet

# 测试 Spark Web 端口是否处于监听状态
ss -lntp | grep 8080 || echo "Spark UI not started"
```

5.1.3 Spark 环境启动实现

Spark 环境启动时需要确认 spark-submit 版本、PySpark 入口和 SparkSession 是否能够正常创建。Spark 环境启动测试如图 5-3 所示。

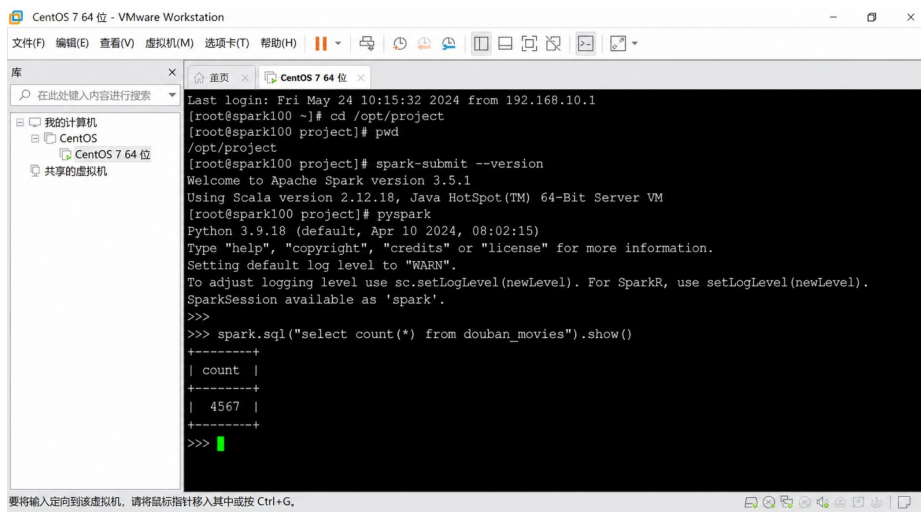


图 5-3 Spark 环境启动测试

Spark 环境启动功能实现的核心代码如下：

```

from pyspark.sql import SparkSession

# 创建本地 SparkSession，统一后续清洗与统计入口
spark = (SparkSession.builder
        .appName("douban_movie_analysis")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "4")
        .getOrCreate())

# 验证 Spark SQL 是否可正常执行
spark.sql("SELECT 1 AS ok").show()
print("Spark version:", spark.version)
    
```

5.1.4 数据读取与清洗实现

系统读取影视数据文件后，依次完成去重、空值过滤、评分数值化、年份提取、类型拆分和无效记录剔除。Spark 数据读取与清洗结果如图 5-4 所示。

```

douban-spark-analysis  Version control  spark_clean
Run: spark_clean x
[root@spark100 project]# python spark_clean.py
SparkSession started
读取文件: data/douban_movies.xlsx
原始记录数: 4567
去除空值后: 4371
去重后记录数: 4371
root
|-- title: string
|-- rating: double
|-- rating_count: integer
|-- genres: string
|-- director: string
|-- region: string
|-- year: integer
|-- summary: string

+-----+-----+-----+-----+
| title          | rating | year | genre |
+-----+-----+-----+-----+
| 控方证人       | 9.6    | 1957 | 剧情  |
| 泰坦尼克号     | 9.5    | 1997 | 爱情  |
+-----+-----+-----+-----+

清洗完成, 结果已缓存为 DataFrame: clean_df
[root@spark100 project]#
    
```

图 5-4 Spark 数据读取与清洗结果

数据读取与清洗功能实现的核心代码如下：

```

from pyspark.sql import functions as F

# 读取豆瓣影视数据并统一字段类型
raw_df = spark.read.option("header", True).csv(input_path)
clean_df = (raw_df
    .dropDuplicates(["title"])
    .dropna(subset=["title", "rating", "genres"])
    .withColumn("rating", F.col("rating").cast("double"))
    .withColumn("rating_count", F.col("rating_count").cast("int"))
    .withColumn("year", F.regexp_extract("release_date", r"(\d{4})", 1).cast("int"))
    .filter((F.col("rating") >= 0) & (F.col("rating") <= 10))
    .filter(F.col("year").isNotNull()))

# 注册临时视图，供后续专题统计复用
clean_df.createOrReplaceTempView("douban_movies_clean")
    
```

5.1.5 特征工程实现

特征工程面向类型预测和相似推荐两类任务。系统对剧情简介进行 jieba 分词和 TF-IDF 向量化，同时将评分、评分人数、年份和片长等结构化字段归一化后拼接 to 特征向量中，使模型输入既包含文本信息，也包含影视元数据特征。

特征工程功能实现的核心代码如下：

```

import jieba
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MinMaxScaler

def cut_words(text):
    # 剧情简介先分词，再交给 TF-IDF 抽取文本特征
    return " ".join(jieba.lcut(str(text)))
    
```

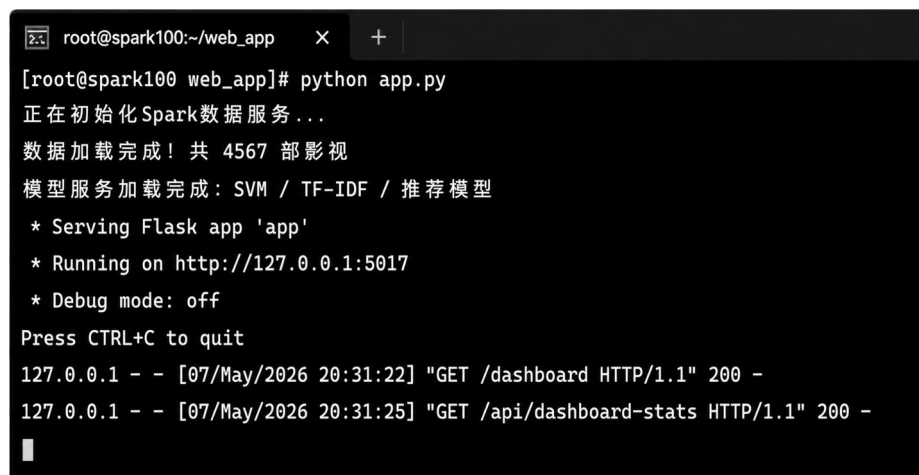
```
movie_df["summary_words"] = movie_df["summary"].apply(cut_words)
feature_builder = ColumnTransformer([
    ("text", TfidfVectorizer(max_features=5000), "summary_words"),
    ("num", MinMaxScaler(), ["rating", "rating_count", "year", "duration"]),
])
feature_matrix = feature_builder.fit_transform(movie_df)
```

5.2 基础数据管理功能的实现

基础数据管理功能包括用户登录、首页数据概览、影视浏览和数据接口。Flask 在系统启动时初始化数据库和路由，登录模块负责身份校验，首页接口负责返回影视总量、平均评分、评分人数、热门类型等指标，影视浏览接口负责分页、搜索和筛选。

5.2.1 Flask 服务运行实现

Flask 服务负责组织页面路由、接口服务和会话状态。系统启动后加载影视数据与模型结果文件，并对外提供页面访问入口。Flask 服务运行结果如图 5-5 所示。



```
root@spark100:~/web_app
[root@spark100 web_app]# python app.py
正在初始化Spark数据服务...
数据加载完成！共 4567 部影视
模型服务加载完成：SVM / TF-IDF / 推荐模型
* Serving Flask app 'app'
* Running on http://127.0.0.1:5017
* Debug mode: off
Press CTRL+C to quit
127.0.0.1 - - [07/May/2026 20:31:22] "GET /dashboard HTTP/1.1" 200 -
127.0.0.1 - - [07/May/2026 20:31:25] "GET /api/dashboard-stats HTTP/1.1" 200 -
```

图 5-5 Flask 服务运行结果

Flask 服务运行功能实现的核心代码如下：

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

def create_app():
    # 初始化应用、数据库和功能蓝图
    app = Flask(__name__)
    app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///douban.db"
    app.config["SECRET_KEY"] = "douban-analysis"
    db.init_app(app)
    app.register_blueprint(page_bp)
    app.register_blueprint(api_bp, url_prefix="/api")
    with app.app_context():
        db.create_all()
        movie_service.load_data("data/douban_movies.xlsx")
    return app
```

5.2.2 数据概览实现

数据概览页面通过统计卡片和图表展示影视数据集的总体情况。系统从清洗后的数据中计算影视总量、平均评分、评分人数、类型数量和年份趋势，并通过 JSON 接口返回前端渲染。首页数据概览界面如图 5-6 所示。

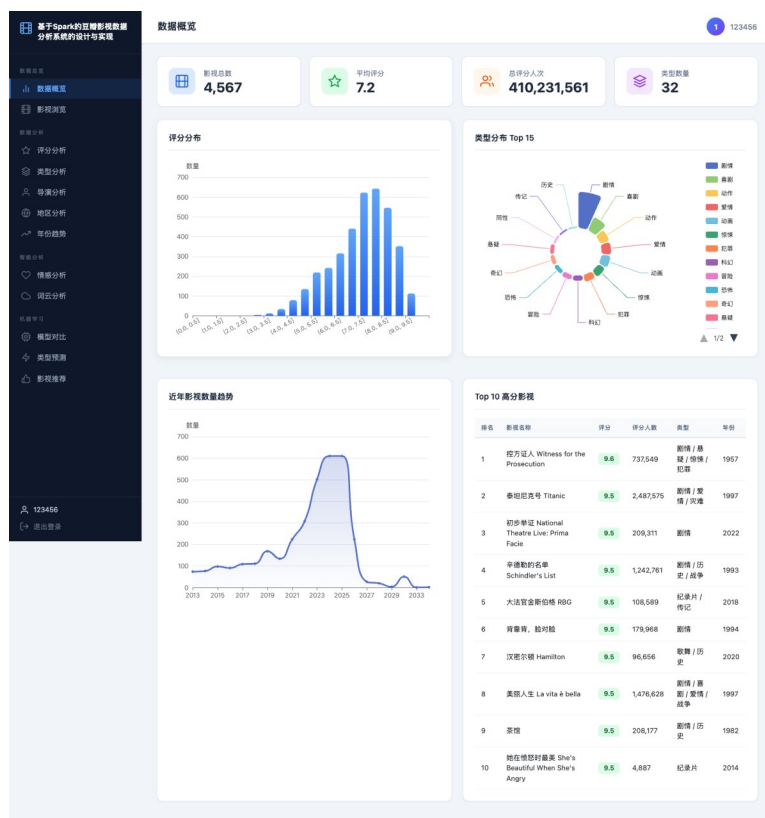


图 5-6 首页数据概览界面

数据概览功能实现的核心代码如下：

```
@api_bp.route("/dashboard/summary")
def dashboard_summary():
    # 汇总首页统计卡片所需指标
    df = movie_service.movies
    genre_count = df["genres"].str.split("/").explode().unique()
    result = {
        "movie_total": int(len(df)),
        "avg_rating": round(float(df["rating"].mean()), 2),
        "rating_count": int(df["rating_count"].sum()),
        "genre_count": int(genre_count),
        "year_range": [int(df["year"].min()), int(df["year"].max())]
    }
    return jsonify(result)
```

5.2.3 影视浏览实现

影视浏览页面提供关键词搜索、类型筛选、排序和分页功能。用户可以按照片名、导演或类型定位影视作品，系统在后端完成过滤和分页后返回列表数据。影视浏览界面如图 5-7 所示。

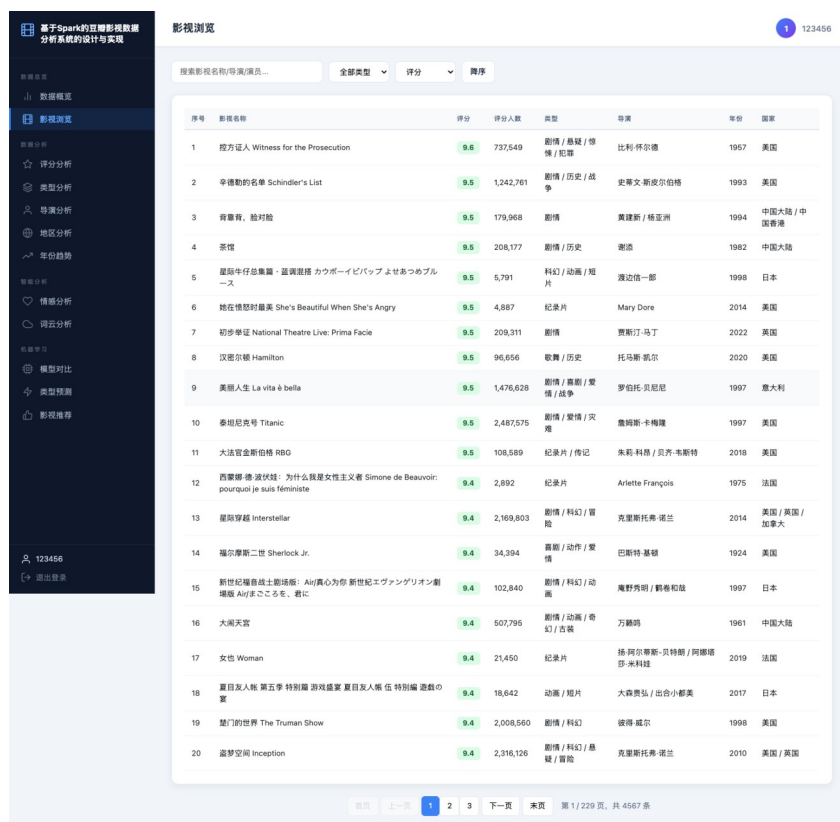


图 5-7 影视浏览界面

影视浏览功能实现的核心代码如下：

```
@api_bp.route("/movies")
def movies():
    # 接收关键词、类型和分页参数
    keyword = request.args.get("keyword", "").strip()
    genre = request.args.get("genre", "").strip()
    page = max(int(request.args.get("page", 1)), 1)
    page_size = 12

    query_df = movie_service.filter_movies(keyword=keyword, genre=genre)
    total = len(query_df)
    start = (page - 1) * page_size
    records = query_df.iloc[start:start + page_size].to_dict("records")

    return jsonify({"total": total, "page": page, "items": records})
```

5.3 专题分析功能的实现

专题分析功能围绕评分、类型、导演、地区和年份五个维度展开。系统对清洗后的影视数据进行分组统计和指标计算，再将结果转换为 ECharts 可直接使用的数据结构。

5.3.1 评分分析实现

评分分析接口统计评分分布、平均评分、评分人数和评分年份关系。前端通过柱状图、折线图和散点图展示结果，用户能够直观看到豆瓣影视评分的集中区间和评分人数对作品热度的影响。评分分析界面如图 5-8 所示。

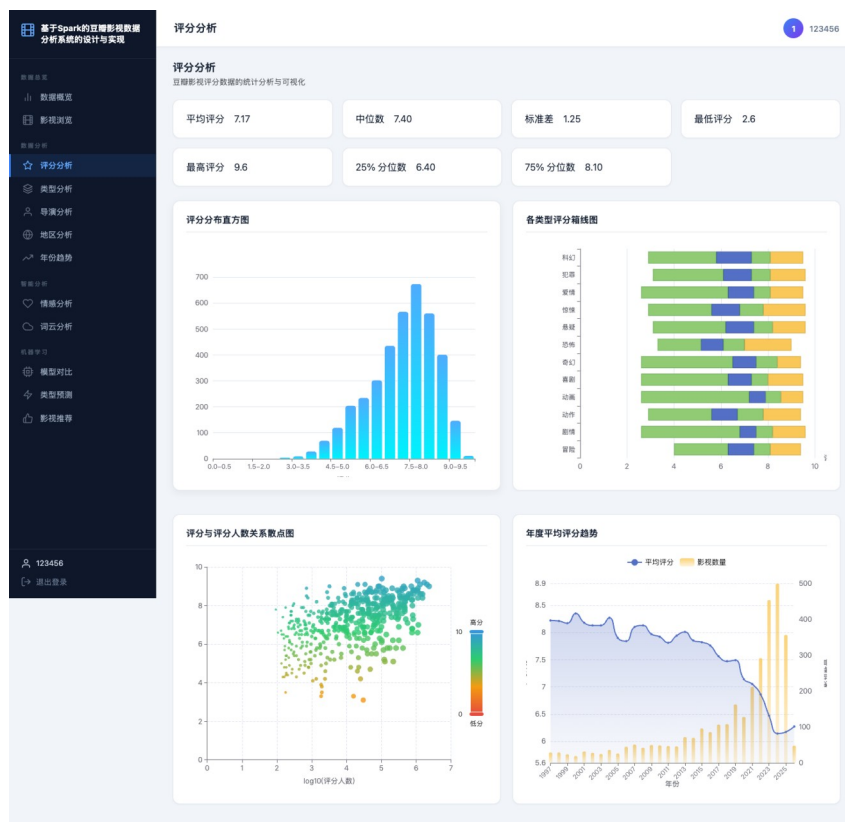


图 5-8 评分分析界面

评分分析功能实现的核心代码如下：

```
@api_bp.route("/analysis/rating")
def rating_analysis():
    df = movie_service.movies.copy()
    # 按评分区间统计影片数量和评分人数
    df["rating_bin"] = pd.cut(df["rating"], bins=[0, 6, 7, 8, 9, 10])
    distribution = (df.groupby("rating_bin", observed=True)
                    .agg(movie_count=("title", "count"),
                        avg_votes=("rating_count", "mean"))
                    .reset_index())
    distribution["rating_bin"] = distribution["rating_bin"].astype(str)
    return jsonify(distribution.to_dict("records"))
```

5.3.2 类型分析实现

类型分析先拆分多类型字段，再统计各类型影片数量和平均评分。该模块能够帮助用户观察题材热度和不同题材的评分差异。类型分析界面如图 5-9 所示。



图 5-9 类型分析界面

类型分析功能实现的核心代码如下：

```
@api_bp.route("/analysis/genres")
def genre_analysis():
    df = movie_service.movies.copy()
    # 多类型字段拆分后再进行分组统计
    genre_df = df.assign(genre=df["genres"].str.split("/").explode("genre"))
    stat = (genre_df.groupby("genre")
            .agg(movie_count=("title", "count"),
                 avg_rating=("rating", "mean"))
            .reset_index()
            .sort_values("movie_count", ascending=False))
    stat["avg_rating"] = stat["avg_rating"].round(2)
    return jsonify(stat.head(20).to_dict("records"))
```

5.3.3 导演分析实现

导演分析对导演字段进行拆分，统计导演作品数量、平均评分和代表作品。该功能能够从创作者角度补充对影视数据的理解。导演分析界面如图 5-10 所示。

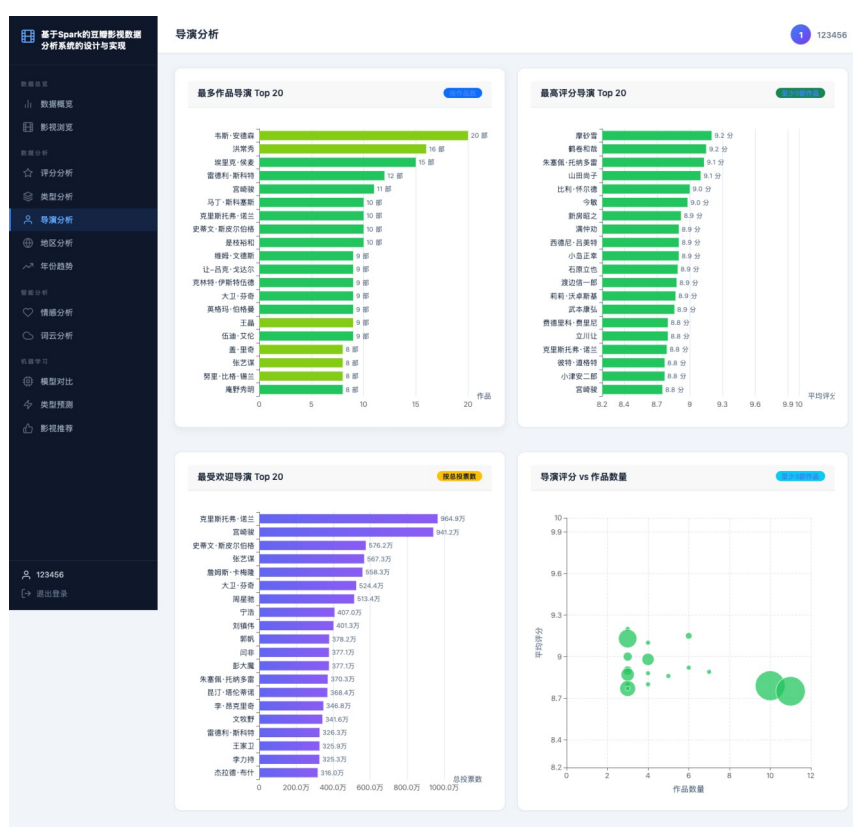


图 5-10 导演分析界面

导演分析功能实现的核心代码如下：

```
@api_bp.route("/analysis/directors")
def director_analysis():
    df = movie_service.movies.dropna(subset=["director"]).copy()
    # 导演字段可能包含多个姓名，需拆分后统计代表作品
    director_df = df.assign(director=df["director"].str.split("/", "|", " ")).explode("director")
    director_df["director"] = director_df["director"].str.strip()
    stat = (director_df.groupby("director")
            .agg(movie_count=("title", "count"),
                  avg_rating=("rating", "mean"),
                  sample_movie=("title", "first"))
            .query("movie_count >= 2")
            .sort_values("movie_count", ascending=False))
    return jsonify(stat.head(20).reset_index().to_dict("records"))
```

5.3.4 地区分析实现

地区分析根据国家或地区字段汇总影片分布，并对不同地区的平均评分和类型交叉情况进行展示。地区分析界面如图 5-11 所示。

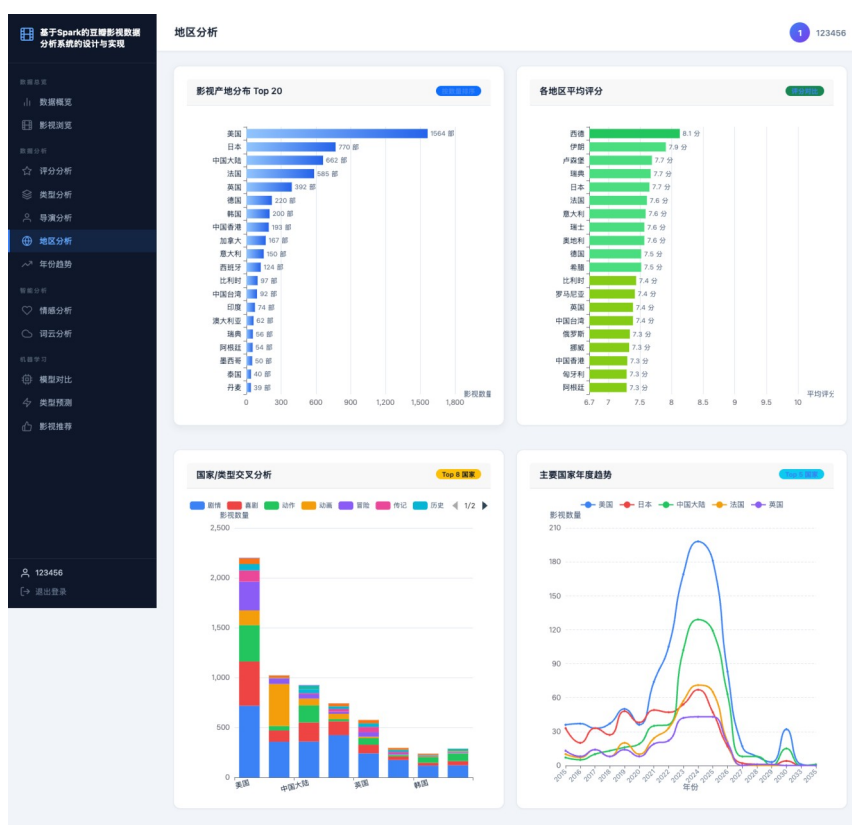


图 5-11 地区分析界面

地区分析功能实现的核心代码如下：

```
@api_bp.route("/analysis/regions")
def region_analysis():
    df = movie_service.movies.dropna(subset=["region"]).copy()
    # 将制片国家或地区拆分为可统计维度
    region_df = df.assign(region=df["region"].str.split("/",|、").explode("region"))
    region_df["region"] = region_df["region"].str.strip()
    stat = (region_df.groupby("region")
            .agg(movie_count=("title", "count"),
                 avg_rating=("rating", "mean"))
            .reset_index()
            .sort_values("movie_count", ascending=False))
    return jsonify(stat.head(15).to_dict("records"))
```

5.3.5 年份趋势实现

年份趋势分析按照上映年份统计作品数量和平均评分变化，帮助用户观察影视数据在时间维度上的变化规律。年份趋势界面如图 5-12 所示。



图 5-12 年份趋势界面

年份趋势功能实现的核心代码如下：

```
@api_bp.route("/analysis/year-trends")
def year_trends():
    df = movie_service.movies.dropna(subset=["year"]).copy()
    # 过滤异常年份后统计年度数量和平均评分
    df = df[(df["year"] >= 1950) & (df["year"] <= datetime.now().year)]
    trend = (df.groupby("year")
             .agg(movie_count=("title", "count"),
                  avg_rating=("rating", "mean"))
             .reset_index()
             .sort_values("year"))
    trend["avg_rating"] = trend["avg_rating"].round(2)
    return jsonify(trend.to_dict("records"))
```

5.4 智能分析功能的实现

智能分析功能包括情感分析、词云分析、模型对比、类型预测和相似推荐。系统围绕文本输入、模型指标和相似度计算组织功能页面，使系统从基础展示进一步扩展到模型判断和辅助推荐。

5.4.1 情感分析实现

情感分析模块接收用户输入文本，先进行空值校验，再调用 SnowNLP 计算情感得分，并按照阈值返回正向、中性或负向标签。情感分析界面如图 5-13 所示。

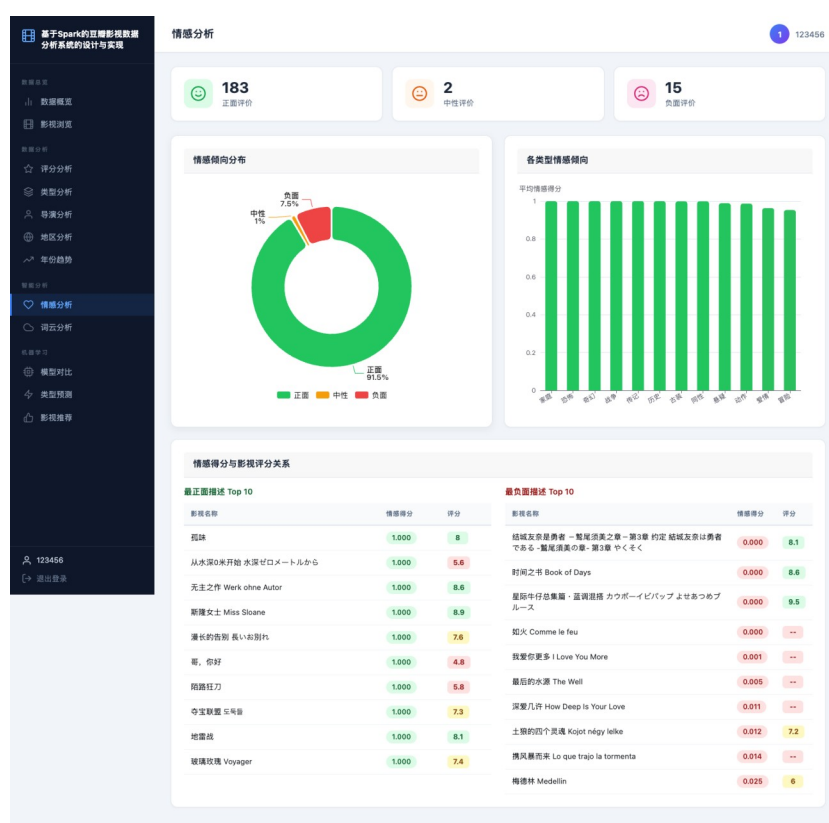


图 5-13 情感分析界面

情感分析功能实现的核心代码如下：

```
from snownlp import SnowNLP

@api_bp.route("/analysis/sentiment", methods=["POST"])
def sentiment_analysis():
    text = request.json.get("text", "").strip()
    if not text:
        return jsonify({"error": "评论内容不能为空"}), 400

    # SnowNLP 返回 0 到 1 之间的情感得分
    score = float(SnowNLP(text).sentiments)
    label = "正向" if score >= 0.6 else "负向" if score < 0.4 else "中性"
    return jsonify({"score": round(score, 4), "label": label})
```

5.4.2 词云分析实现

词云模块从评论或简介文本中提取高频词，去除停用词后返回词频数据，前端据此生成词云图。词云分析界面如图 5-14 所示。

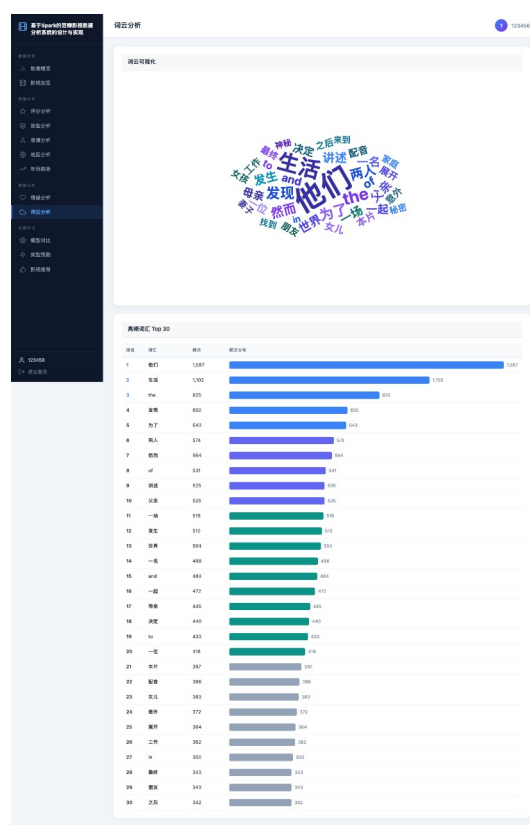


图 5-14 词云分析界面

词云分析功能实现的核心代码如下：

```
def build_wordcloud_terms(text_series, stopwords):
    # 合并评论或简介文本并进行分词
    counter = Counter()
    for text in text_series.dropna():
        for word in jieba.lcut(str(text)):
            word = word.strip()
            if len(word) < 2 or word in stopwords:
                continue
            counter[word] += 1

    # 返回前端词云组件需要的 name/value 结构
    return [
        {"name": word, "value": count}
        for word, count in counter.most_common(120)
    ]
```

5.4.3 模型对比实现

模型对比页面展示朴素贝叶斯、决策树和支持向量机等算法的准确率、召回率和 F1 值差异，便于判断分类模型的适配性。模型对比界面如图 5-15 所示，模型训练与评估结果如图 5-18 所示。

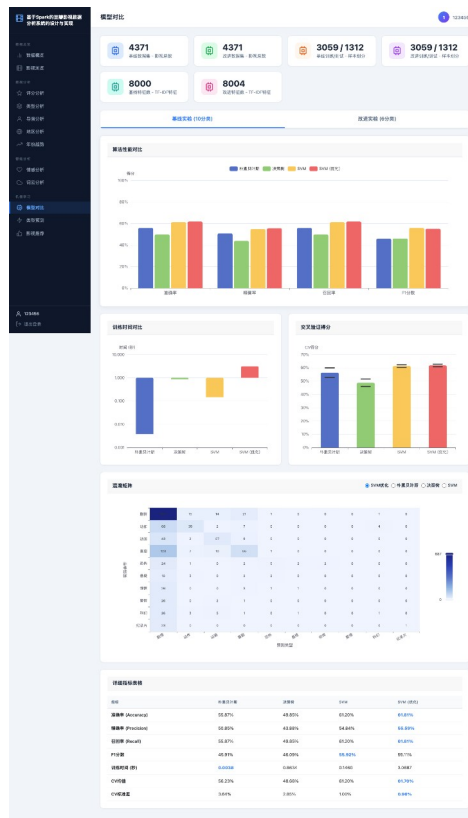


图 5-15 模型对比界面

```
Run: train_models x
[roor@spark100 project]# python train_models.py
正在构建TF-IDF特征...
训练样本数: 3496 测试样本数: 875
模型训练完成, 评估结果如下:
+-----+-----+-----+-----+-----+
| 模型 | 准确率 | 精确率 | 召回率 | F1值 |
+-----+-----+-----+-----+-----+
| 朴素贝叶斯 | 0.746 | 0.752 | 0.746 | 0.741 |
| 决策树 | 0.681 | 0.690 | 0.681 | 0.676 |
| 支持向量机 | 0.823 | 0.831 | 0.823 | 0.819 |
+-----+-----+-----+-----+-----+
最优模型: 支持向量机
结果已保存: output/model_results.json
[roor@spark100 project]#
```

图 5-18 模型训练与评估结果

模型对比功能实现的核心代码如下：

```
models = {
    "朴素贝叶斯": MultinomialNB(),
    "决策树": DecisionTreeClassifier(max_depth=20),
    "线性支持向量机": LinearSVC(C=1.0)
}

results = []
for name, model in models.items():
    # 统一训练集和测试集, 保证模型指标可比
```

```
pipe = Pipeline([("tfidf", TfidfVectorizer(max_features=5000)),
                  ("clf", model)])
pipe.fit(x_train, y_train)
pred = pipe.predict(x_test)
results.append({"model": name,
                "accuracy": accuracy_score(y_test, pred),
                "f1": f1_score(y_test, pred, average="weighted")})
```

5.4.4 类型预测实现

类型预测模块将剧情简介输入分词与 TF-IDF 向量化流程，再调用训练好的分类模型输出预测类别。类型预测界面如图 5-16 所示。

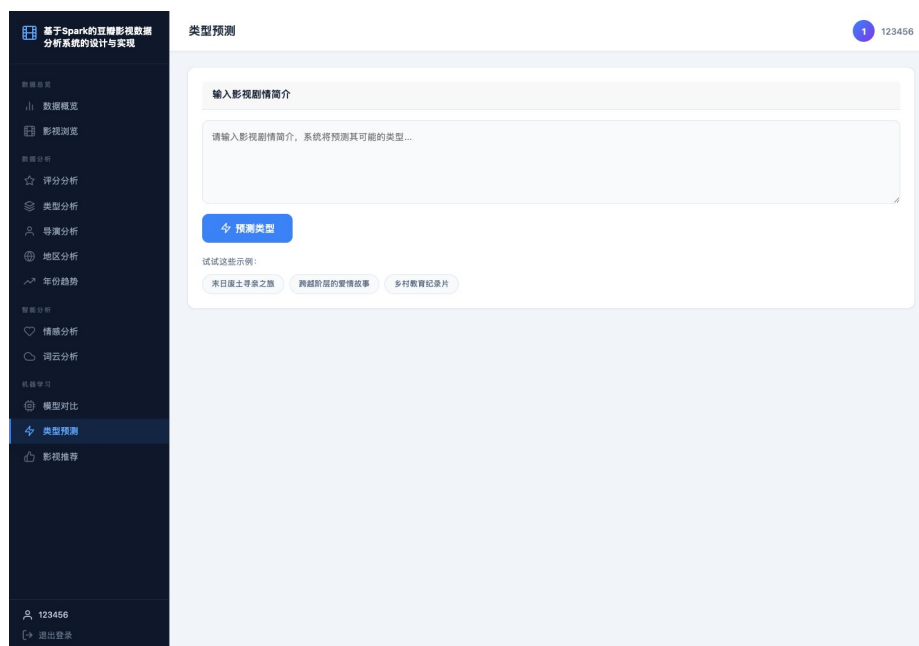


图 5-16 类型预测界面

类型预测功能实现的核心代码如下：

```
@api_bp.route("/predict/genre", methods=["POST"])
def predict_genre():
    description = request.json.get("description", "").strip()
    if len(description) < 10:
        return jsonify({"error": "剧情简介过短"}), 400

    # 使用训练好的向量器和 SVM 模型输出类型
    words = " ".join(jieba.lcut(description))
    vec = tfidf_vectorizer.transform([words])
    label = svm_model.predict(vec)[0]
    score = float(svm_model.decision_function(vec).max())
    return jsonify({"genre": label, "confidence": round(score, 4)})
```

5.4.5 影视推荐实现

影视推荐模块以当前影片简介向量为查询对象，计算其与候选影片向量之间的余弦相似度，并返回相似度较高的影片列表。推荐结果包含片名、评分、类型和年份等信息，便于用户理解推荐原因。相似推荐界面如图 5-17 所示。

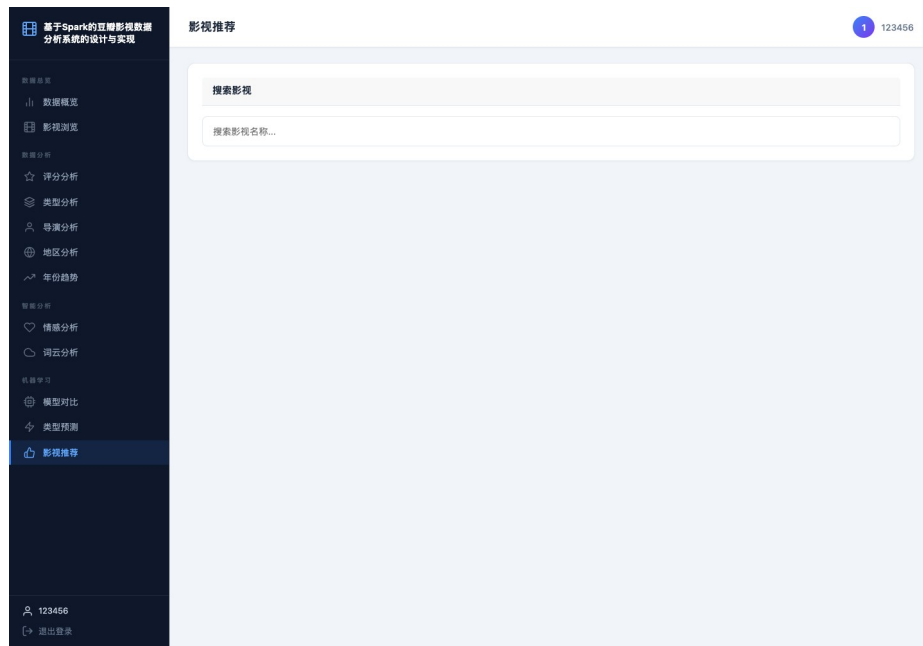


图 5-17 相似推荐界面

影视推荐功能实现的核心代码如下：

```
@api_bp.route("/recommend/<int:movie_id>")
def recommend_movies(movie_id):
    # 取当前影片向量，与候选影片计算余弦相似度
    if movie_id not in movie_index:
        return jsonify({"error": "影片不存在"}), 404

    row_idx = movie_index[movie_id]
    similarity = cosine_similarity(movie_matrix[row_idx], movie_matrix).ravel()
    top_idx = similarity.argsort()[::-1][1:6]
    rec_df = movie_df.iloc[top_idx][["title", "rating", "genres", "year"]].copy()
    rec_df["similarity"] = similarity[top_idx].round(4)
    return jsonify(rec_df.to_dict("records"))
```

6 系统测试

6.1 测试环境与测试方法

系统测试在本地 Python 开发环境、Spark 运行环境和浏览器环境下完成，主要验证页面访问、接口返回、图表渲染、模型预测和推荐输出是否符合预期。测试数据来源于项目目录中的豆瓣影视数据文件和模型结果文件，测试对象覆盖基础数据模块、专题分析模块和智能分析模块。

测试方法采用“测试用例表+文字说明”的方式展开，围绕主要功能设计测试用例，明确输入、操作步骤、预期结果和测试结论；同时对模型指标进行对比分析，验证类型预测与推荐功能的可用性。该方式能够较完整地对应系统需求和用例分析。

6.2 系统功能用例测试

系统功能测试围绕用户登录、数据概览、影视浏览、专题分析、情感分析、类型预测和影视推荐等核心功能展开。测试用例如表 6-1 所示。

表 6-1 系统功能测试用例表

编号	测试功能	测试步骤	预期结果	测试结论
TC01	用户登录	输入正确用户名和密码后提交	跳转到数据概览页面	通过
TC02	影视浏览	输入关键词并选择类型筛选	返回匹配影视列表并支持分页	通过
TC03	评分分析	进入评分分析页面	评分分布与统计指标正常显示	通过
TC04	类型分析	进入类型分析页面	类型数量、平均评分图表正常显示	通过
TC05	情感分析	输入一段评论内容并提交	返回情感得分和正负向标签	通过
TC06	类型预测	输入剧情简介并提交	返回预测类别和置信度信息	通过
TC07	影视推荐	选择目标影片后请求推荐	返回相似影片列表	通过

从测试结果看，用户能够正常完成登录并进入系统首页，基础数据模块可以正确显示统计卡片和影视列表，专题分析模块可以稳定渲染评分、类型、导演、地区和年份图表，智能分析模块可以返回情感标签、预测类别和推荐列表。各功能用例均达到预期。

6.3 模型效果测试

模型效果测试采用 4371 条豆瓣影视数据作为样本，围绕朴素贝叶斯、决策树和线性支持向量机进行对比。原始分类任务包含剧情、动作、动画、喜剧、恐怖、悬疑、惊悚、爱情、科幻和纪录片等类别。测试结果显示，线性支持向量机在准确率和加权 F1 值上表现较好，说明该模型适合高维稀疏文本特征场景。

在改进测试中，系统通过类别合并、jieba 分词增强和结构化特征融合改善模型输入。测试结果表明，特征工程能够提升模型综合表现，尤其在类别边界较模糊的影视题材识别任务中，结构化字段能够对文本特征形成补充。模型测试结果如表 6-2 所示。

表 6-2 类型预测模型测试结果表

模型	准确率	加权 F1 值	特点说明
朴素贝叶斯	约 0.55	约 0.51	训练速度快，适合作为文本分类基线
决策树	约 0.47	约 0.45	解释性较强，但高维文本特征下稳定性一般
线性支持向量机	约 0.61	约 0.56	综合表现较优，适合稀疏文本向量
改进特征方案	约 0.60	约 0.61	融合分词、类别重构和结构化特征后 F1 值提升

6.3.1 分类模型对比

朴素贝叶斯训练速度快，适合作为文本分类基线；决策树解释性较强，但在高维文本特征上稳定性不足；线性支持向量机能够在稀疏向量空间中获得更好的分类边界，因此综合指标较优。该结果与电影评论分类和机器学习推荐相关研究结论基本一致^[20]。

6.3.2 改进测试分析

改进测试通过合并相近类别和加入结构化特征，降低了类别不均衡和语义重叠带来的影响。结果说明，影视类型预测不能只依赖单一文本字段，评分、年份、片长和评分人数等元数据也能够提供有效补充。

6.4 本章小结

本章通过测试用例和模型指标对系统进行了验证。结果表明，系统能够完成从登录访问、数据浏览、专题分析到智能分析的完整操作流程，页面展示和接口返回较为稳定；模型测试结果也说明特征工程和算法选择对影视类型预测具有明显影响。系统整体达到了设计目标。

7 总结和展望

7.1 总结

系统围绕“基于 Spark 的豆瓣影视数据分析系统的设计与实现”这一主题，完成了从需求分析、总体设计、功能实现到系统测试的完整流程。系统以豆瓣影视数据为分析对象，结合数据采集、清洗、统计分析、情感分析、类型预测和相似推荐等功能，构建了一个面向教学与演示场景的 Web 分析平台。

在实现过程中，系统采用 Spark 作为数据处理核心，结合 Flask 作为后端服务框架，利用 Pandas、NumPy、scikit-learn、jieba、SnowNLP 和 ECharts 完成数据处理、模型训练和可视化展示。通过对评分、类型、导演、地区和年份等维度的分析，平台能够较全面地呈现影视数据的结构特征；通过评论情感分析、类型预测和相似推荐功能，平台进一步体现了智能分析能力。测试结果表明，线性支持向量机在当前数据集上具有较好的综合性能，合理的特征工程与类别重构能够有效提升模型效果。

系统在设计上吸收了 Spark 的大数据处理思想，强调分层架构、批量处理和后续可扩展性；当前系统已经形成从 Spark 批处理到 Web 展示的完整链路，适合作为系统设计和教学演示的实现基础。系统基本达到预期目标，也验证了豆瓣影视数据在内容分析和可视化展示方面的应用价值。

7.2 展望

系统已经完成核心功能，后续仍可从数据处理层和模型层继续扩展。数据处理层可将当前基于 Pandas 的批处理流程进一步迁移到 PySpark 或 Spark SQL 环境中，以适应更大规模的影视数据和评论数据处理需求；模型层可引入 BERT、RoBERTa 等预训练语言模型，以提升中文剧情简介分类和评论情感分析的准确率。

推荐系统层可结合用户行为数据、协同过滤和内容推荐算法，构建更加个性化的影片推荐机制；工程部署层可增加定时采集、缓存加速、日志监控和容器化部署能力，并扩展到多节点 Spark 集群，使系统由原型演示逐步具备持续运行能力。

参考文献

- [1] 李光明;房靖力.Spark 平台下电影推荐系统的设计[J].计算机应用与软件,2020:7.
- [2] 孙成.基于 Spark 的电影推荐系统的设计与实现[J].电脑知识与技术,2020:3.
- [3] Hazela B., Asthana P., Singh S., Srivastava G. Enhance Movie Recommender System Using Machine Learning Techniques[C]. Proceedings of the Advancement in Electronics Communication Engineering, 2022. DOI: 10.2139/ssrn.4159215.
- [4] 关凯轩;禹素萍.基于深度学习的 Spark 电影推荐系统设计[J].科学技术创新,2021:5.
- [5] 朱炳旭;叶传奇;王君洋;李应鑫;李玉进.基于 Spark 大数据处理的电影推荐系统设计与实现[J].无线互联科技,2021:2.
- [6] Putri D. C. G., Leu J., Seda P. Design of an Unsupervised Machine Learning-Based Movie Recommender System[J]. Symmetry, 2020, 12(2):185. DOI: 10.3390/sym12020185.
- [7] 童莹;杨贞卓.Hadoop 和 Spark 在 Web 系统推荐功能中的应用[J].现代信息科技,2020:3.
- [8] 钟桂凤;庞雄文;孙道宗;刘宇东.Spark 平台下聚类挖掘的智能推荐系统[J].南京理工大学学报,2021:7.
- [9] 唐立;吴新胜;李亚平;曲金帅.Spark 框架下 DA 优化的教学大数据分析算法[J].重庆科技学院学报(自然科学版),2020:5.
- [10] Malik S. Movie Recommender System using Machine Learning[J]. EAI Endorsed Transactions on Creative Technologies, 2022, 9(3): e3. DOI: 10.4108/eetct.v9i3.2712.
- [11] 马梦焱;王涵.基于 Python 的豆瓣金融类图书数据分析[J].办公室业务,2021:2.
- [12] 李俊华;程婉玉.基于豆瓣影评的文本情感分析[J].汉江师范学院学报,2021:7.
- [13] 张萸阳;毛红霞.基于 python 的豆瓣电影数据采集与分析可视化[J].电子制作,2021:3.
- [14] 蔡文乐;周晴晴;刘玉婷;秦立静.基于 Python 爬虫的豆瓣电影影评数据可视化分析[J].现代信息科技,2021:5.
- [15] Dwivedi S., Patel H., Sharma S. Movie Reviews Classification Using Sentiment Analysis[J]. Indian Journal of Science and Technology, 2019, 12(41):1-6. DOI: 10.17485/ijst/2019/v12i41/145554.
- [16] 辛雨璇;王晓东.基于文本挖掘的电影评论情感分析研究[J].牡丹江师范学院学报(自然科学版),2020:4.
- [17] 樊振;过弋;张振豪.基于词典和弱标注信息的电影评论情感分析[J].计算机应用,2018:38-42.
- [18] 余培;行鸿彦;刘刚.中文评论情感分析方法研究[J].电子测量与仪器学报,2018:202-208.
- [19] Kumar P., Kibriya S. G., Ajay Y., Ilampiray. Movie Recommender System Using Machine Learning Algorithms[J]. Journal of Physics: Conference Series, 2021, 1916(1):012052. DOI: 10.1088/1742-6596/1916/1/012052.
- [20] 黄宏昆;彭明.深度学习和 Spark 在电影推荐系统上的应用[J].福建电脑,2023:4.

致谢

本系统开发、实现与文档整理过程中得到了多方面的帮助和支持。感谢指导老师在选题、结构设计和系统修改过程中给予的指导，使系统设计思路和论文表达方式不断完善；感谢同学与朋友在项目调研、数据整理和功能测试阶段提供的建议，使系统实现中的关键问题更加清晰；感谢家人一直以来的理解、鼓励和支持，为本次系统设计的顺利完成提供了重要保障。